



*Debre Berhan university
Collage of Compuing
Departement of computer
science*

*PROJECT
MANAGEMEMNT*

COURSE CODE: COSC₁₀₁₂

ASSIGNMENT

*prepared by:
Makbel kebede
ID: 1601325*

Submitted to: Ms Tigst

submission date: 26/08/2018 E.C

1. Assignment one

1.1. Brief history of Software Project Management (SPM) with different perspectives?

Early Roots (1950s–1960s) – The “Software Crisis”

In the early days of computing, software was developed without formal management. Projects were often late, over budget, or failed entirely.

- **1968 NATO Conference** coined the term “*software crisis*” – systems were too complex, schedules unrealistic, quality poor.
- No standard processes: developers simply coded until something worked.

1970s – Birth of Structured Methods

- **Waterfall model** (Winston Royce, 1970) introduced phases: requirements → design → implementation → testing → maintenance.
- This brought **planning and control** into software projects.
- Project management focused on **documentation, milestones, and phase completion**.

1980s – Emergence of PM as a Discipline

- **PMI (Project Management Institute)** founded 1969, but gained traction in 1980s.
- **PMBOK Guide** first published 1987 (though slides reference later editions).
- Software projects began using **cost estimation models** (COCOMO – Barry Boehm, 1981).
- Risk management became recognized (Boehm’s *Software Risk Management*, 1989).

1990s – Process Maturity and Agile Beginnings

- **CMM (Capability Maturity Model)** from SEI – focused on improving software processes.
- Heavyweight methodologies (RUP, MSF) emphasized discipline.
- Late 1990s: frustration with bureaucracy led to **Agile Manifesto (2001)** – though its roots are in 1990s iterative methods (Scrum, XP).

2000s–Present – Hybrid and Adaptive Approaches

- Agile became mainstream (Scrum, Kanban, SAFe).
- Integration of **project management with DevOps, CI/CD**.

- Modern SPM balances **predictive planning** (Waterfall) with **adaptive execution** (Agile).
- Tools evolved: Jira, MS Project, Trello, Asana – supporting virtual and distributed teams.

Different Perspectives on SPM History

Perspective	Key Focus	Example
Technical	Process models, estimation, risk analysis	Boehm's spiral model
Managerial	Planning, control, resource allocation	PMBOK framework
Human	Team motivation, leadership, communication	Agile's people-over-process
Quality	Defect prevention, standards, maturity	CMMI, Six Sigma
Business	ROI, strategic alignment, portfolio management	Project portfolio management (Ch.1 slides)

Generally SPM evolved from chaotic coding to a disciplined, multi-perspective field that now integrates technical, human, and business considerations.

1.2. Discussion on SDLC & Methodologies

What is SDLC?

Software Development Life Cycle (SDLC) is a framework defining phases to build software. From **Chapter 1 (Lesson 2)** and **Chapter 3 (Triple Constraints)**, common phases:

1. Requirements
2. Design
3. Implementation (coding)
4. Testing
5. Deployment
6. Maintenance

Predictive Methodologies (Plan-Driven)

- **Waterfall** – linear, heavyweight documentation.

- Best when requirements are stable, low uncertainty.
- Example: large government or safety-critical systems.
- **V-Model** – each development phase has a corresponding testing phase.
 - Emphasizes verification and validation.

Adaptive Methodologies (Agile)

- **Scrum** – iterative sprints (2–4 weeks), roles: Product Owner, Scrum Master, Team.
 - Daily standups, sprint reviews, retrospectives.
- **Kanban** – continuous flow, visualized on boards (Ch.5 communications slide).
- **Extreme Programming (XP)** – test-driven development, pair programming, frequent releases.

Hybrid Approaches

- **Water-Scrum-Fall** – planning and requirements (waterfall) → development (scrum) → deployment (waterfall).
- **SAFe (Scaled Agile Framework)** – agile at scale for large enterprises.

Considerations When Choosing a Methodology (from Ch.1, Ch.4)

Factor	Predictive (Waterfall)	Adaptive (Agile)
Requirements clarity	High, stable	Evolving, uncertain
Project size	Large, complex	Small to medium
Customer involvement	Low (milestones only)	High (continuous)
Risk tolerance	Low (plan to avoid)	High (inspect and adapt)
Quality approach	Heavy testing at end	Built-in, continuous
Documentation	Extensive	Just enough

From Ch.4 quality perspective: Adaptive models prefer *prevention over inspection* (modern quality management). Predictive models rely on *phase-end inspections*.

1.3. Real-life project success & failure scenario analysis

Case Study: Denver International Airport (DIA) Baggage Handling System

(Selected because it is a classic software-intensive project failure covered in project management literature – aligns with “problems with software projects” in Ch.1 slides)

Background:

- 1990s – fully automated baggage system using networked software, sensors, carts.
- Designed to handle 4,000 carts, 1,000 destinations.

Why it failed (software project perspective):

Problem	Mapping to Ch.1 issues
Requirements changed constantly	“Change is tempting” (Ch.1 slide 20)
Overoptimistic schedule	Young engineers/optimism (Ch.1 slide 19)
New technology untested	Rapid tech change introduces liability (Ch.1 slide 19)
Poor integration management	“Key to success = good integration management” (Ch.2 slide 4)
No formal change control	Ch.2 change control board missing
Scope creep	Ch.3 scope management ignored

Result:

- Opened 16 months late.
- Cost overrun: from 1.7B to 4.8B.
- System never fully worked – eventually partially dismantled.

Success scenario comparison – London Heathrow Terminal 5 (different domain but software-managed baggage):

- Used **extensive simulation, staged rollout, rigorous testing** before go-live.
- Success factor: strong project integration management and stakeholder involvement (Ch.2, Ch.5).

Lessons Learned for :

1. **Do not start without clear, stable requirements.**
2. **Use change control processes**
3. **Involve stakeholders early and often**
4. **Manage risk proactively**

5. **Select methodology based on project uncertainty** – DIA needed more iterative testing, not pure waterfall.
-

Summary Assignment one

Sub-question	Key Takeaway
1.1 History of SPM	From software crisis (1960s) → Waterfall → PMBOK → CMM → Agile → Hybrid
1.2 SDLC & Methodologies	Predictive (Waterfall, V-model) vs. Adaptive (Scrum, Kanban) vs. Hybrid; choice depends on uncertainty and stakeholder needs
1.3 Real project analysis	DIA baggage system failed due to poor scope, change, and integration management; success requires formal processes

2.Assignment two

Understanding Schedule Management Concepts

2.1. Lead Time, Lag Time, Slack/Float Time – Definitions with Examples

2.1.1 Lead Time

Definition:

Lead time is the amount of time by which a successor activity can **advance** relative to its predecessor. It allows overlapping of activities.

Sequencing activities involves evaluating leads and lags.

Example:

- Activity A: Write software design (10 days)
- Activity B: Start coding
- If coding can begin **3 days before** design finishes → **Lead time = 3 days**

- Dependency: Start-to-Start with 3-day lead (B starts 3 days after A starts, but A still continues)

Project Management Formula:

In MS Project or network diagrams, lead time is represented as a **negative lag** (e.g., -3 days).

When to use lead time:

- To compress schedule (fast-tracking)
- When partial completion of predecessor allows successor to begin
- Example from Chapter 2 (DNA Sequencing Project): hiring technical replacement while still developing scope statement

2.1.2. Lag Time

Definition:

Lag time is a **delay** between the finish of one activity and the start of its successor. It forces a waiting period.

From Chapter 3, page 50 – Leads and lags are critical in sequencing.

Example:

- Activity A: Pour concrete foundation (5 days)
- Activity B: Frame walls
- Concrete must **cure for 3 days** before framing → **Lag time = 3 days**
- Dependency: Finish-to-Start with 3-day lag

Software project example:

- Activity A: Complete unit testing (7 days)
- Activity B: Begin integration testing
- Wait 2 days for test environment reset → Lag = 2 days

Visual representation (from Chapter 3, page 53 – Task Dependency Types):

Lag is a positive value added to the dependency.

Dependency Type	With Lag	Meaning
FS + 2 days	A finishes → wait 2 days → B starts	
SS + 3 days	A starts → wait 3 days → B starts	

2.1.3 Slack / Float Time

Definition (from standard PM practices, referenced in Chapter 3 CPM section):

Slack (or float) is the amount of time an activity can be delayed **without delaying** the project finish date.

Two types:

Type	Definition	Who controls
Total Float	Time an activity can be delayed without delaying project completion	Project manager
Free Float	Time an activity can be delayed without delaying successor's early start	Activity owner

Calculation (from Chapter 3, Critical Path Method examples):

- **Total Float = Late Start – Early Start**
- **Or Total Float = Late Finish – Early Finish**

Example from Chapter 3, page 62 (Project X):

Path	Duration	Critical?
A-D-H-J	14 days	No
B-E-H-J	16 days	Yes (Critical)
B-F-J	9 days	No
C-G-I-J	14 days	No

- Critical path = longest path = 16 days (B-E-H-J)
- Activities on critical path have **zero float**
- Activity A (path 1): Float = 16 – 14 = 2 days
- Activity F (path 3): Float = 16 – 9 = 7 days

Why slack matters (from Chapter 3, page 67 – Schedule Control):

- Negative float = project behind schedule (requires corrective action)

- Positive float = schedule flexibility (resources can be shifted)

Rule of thumb (from slide 110 – Schedule Performance):

“If SV (Schedule Variance) is negative → behind schedule. Activities with zero float are critical.”

Summary Table: Lead, Lag, Slack

Term	Direction	Effect on Schedule	Example
Lead	Accelerates	Shortens project	Coding starts before design finishes
Lag	Delays	Lengthens project	Wait after concrete pour
Slack/Float	Buffer	Allows delay without affecting end date	Activity A can start 2 days late

2.2. Agile and Schedule Management?

Traditional vs. Agile Scheduling (from Chapter 1 & Chapter 3)

Aspect	Traditional (Predictive)	Agile (Adaptive)
Schedule basis	Detailed Gantt chart, CPM	Sprint backlog, story points
Planning horizon	Full project upfront	Per iteration (2–4 weeks)
Change handling	Formal change control board (Ch.2)	Changes can enter next sprint
Milestones	Fixed dates (phase exits)	Sprint reviews, releases
Critical path	Calculated via network diagram	Not used – focus on velocity

Key Agile Scheduling Concepts

1. Timeboxing

- Each sprint has a **fixed duration** (e.g., 2 weeks).
- Scope is adjusted; date is not moved.
- Example: If team cannot finish all stories → lower priority stories move to next sprint.

2. Velocity-Based Scheduling

- Measure how many story points completed per sprint (historical velocity).
- Forecast release date:
$$\text{Total story points} / \text{Average velocity} = \text{Number of sprints}$$

3. Sprint Backlog & Daily Standups

- Schedule is managed **daily**, not weekly or monthly.
- Burndown charts show remaining work vs. time.

4. No Critical Path – Instead, “Flow”

- Kanban uses **Lead Time** (from request to delivery) and **Cycle Time** (start to finish).
- WIP (Work in Progress) limits prevent bottlenecks.

5. Release Planning vs. Project Schedule

- Agile schedule is **rolling wave** – detailed for current sprint, high-level for future sprints.
- From Chapter 2 (progressive elaboration): *“Scope becomes more clear and specific over time.”*

Example Comparison – Building a Mobile App (4 months)

Activity	Waterfall Schedule	Agile Schedule
Requirements	3 weeks upfront	User stories written each sprint
Design	2 weeks	Just-in-time design
Coding	6 weeks	Across 6 sprints
Testing	3 weeks at end	Every sprint (automated tests)
Change request	Formal CCB approval	Added to product backlog for prioritization

2.3 How the schedule of an agile project will be managed

From best practices aligned with Chapter 3 and Chapter 5 (HRM/Communication):

Step-by-Step Agile Schedule Management Process

Step 1 – Product Backlog Refinement

- Product Owner prioritizes features.
- No dates assigned yet – only order.

Step 2 – Sprint Planning (Schedule for next 2–4 weeks)

- Team selects stories from top of backlog.
- Tasks are broken down (4–16 hours each).
- **Commitment:** Team agrees to complete selected stories.

Step 3 – Daily Schedule Tracking

- Daily standup (15 minutes):
 - What did I complete yesterday?
 - What will I do today?
 - Any blockers?
- **Burndown chart** updated daily (remaining hours vs. days in sprint).

Step 4 – Mid-Sprint Adjustment

- If behind schedule → team removes low-priority tasks (scope sacrifice, not date change).
- No one adds work without removing equivalent effort.

Step 5 – Sprint Review (Schedule demo)

- Show completed work to stakeholders.
- Get feedback that may affect future sprint scheduling.

Step 6 – Sprint Retrospective (Process improvement)

- Team asks: *Why were we behind/ahead?*
- Adjust velocity estimates for next sprints.

Step 7 – Release Management

- Project schedule is a **release burndown chart**:
 - X-axis = sprints
 - Y-axis = remaining story points
- Forecast completion sprint based on average velocity.

Tools for Agile Schedule Management

Tool	Purpose
Jira/VersionOne	Sprint backlog, burndown charts
Kanban board (physical or digital)	Visualize WIP, cycle time
MS Project (Agile mode)	Hybrid scheduling
Shared calendar	Team availability (resource calendars – Ch.5)

Key Differences Summary Table

Traditional Schedule Management	Agile Schedule Management
Fixed plan, variable results	Fixed sprint length, variable scope
Gantt charts updated monthly	Burndown charts updated daily
Critical path analysis	Velocity and flow metrics
Change requests take days/weeks	Changes accepted every sprint
Manager pushes schedule	Team pulls work

Summary of assignment 2

Sub-question	Key Takeaway
2.1 Lead, Lag, Slack	Lead = accelerate (negative lag), Lag = delay (positive wait), Slack = buffer time on non-critical paths
2.2 Agile vs. Traditional schedule	Agile uses timeboxing, velocity, rolling wave; no critical path
2.3 Agile schedule management	Daily burndowns, sprint planning, release forecasting, continuous adjustment

Assignment Question 3

How EVM is used to track the performances of triple constraints of a project (Scope, Time, & Cost)

What is Earned Value Management (EVM)?

“EVM is a project performance measurement technique that integrates the triple constraints to objectively assess how a software project is progressing compared to the plan.”

EVM answers three fundamental questions for any point in time:

Question	EVM Metric
Where should we be?	Planned Value (PV)
Where are we actually?	Earned Value (EV)
What did we spend?	Actual Cost (AC)

How EVM Tracks Each of the Triple Constraints

Triple Constraint	EVM Metric	Formula	Interpretation
Scope	EV (Earned Value)	$EV = \% \text{ Complete} \times BAC$	Measures how much scope is <i>actually</i> completed in budget terms, not just planned.
Time (Schedule)	SV (Schedule Variance) & SPI (Schedule Performance Index)	$SV = EV - PV$ $SPI = EV / PV$	Negative SV or $SPI < 1 \rightarrow$ behind schedule. Positive SV or $SPI > 1 \rightarrow$ ahead of schedule.
Cost	CV (Cost Variance) & CPI (Cost Performance Index)	$CV = EV - AC$ $CPI = EV / AC$	Negative CV or $CPI < 1 \rightarrow$ over budget. Positive CV or $CPI > 1 \rightarrow$ under budget.

Example:

Project: Complete 7 modules in 7 days, budget = 700(100 per module)

At end of day 3 (planned):

- $PV = 3 \text{ modules} \times 100 = 300$

Reality at end of day 3:

- Only 2 modules completed
- Actual cost incurred = \$250

Calculations:

Metric	Value	Meaning
$EV = 2 \times \$100$	\$200	Only \$200 worth of scope actually completed
AC	\$250	We spent 250 <i>to get</i> 200 worth of work
$SV = 200 - 300$	-\$100	Behind schedule (negative)
$SPI = 200/300$	0.67	$< 1 \rightarrow$ behind schedule
$CV = 200 - 250$	-\$50	Over budget (negative)
$CPI = 200/250$	0.8	$< 1 \rightarrow$ over budget

- $SPI < 1$ and SV negative $\rightarrow$ **schedule problem**
- $CPI < 1$ and CV negative $\rightarrow$ **cost problem**
- Both negative $\rightarrow$ project is both late and over budget (common in software projects)

How EVM Integrates All Three Constraints

A single EVM analysis answers:

Question	Constraint	Answer from EVM
Are we delivering what we planned?	Scope	EV compared to PV and BAC
Are we on time?	Time	SV and SPI
Are we on budget?	Cost	CV and CPI

“EVM is used mainly for Schedule & Cost” but the scope baseline (BAC) is the foundation.

Additional EVM Terms for Complete Tracking (Chapter 3, page 113–114)

Term	Definition	Use
BAC (Budget at Completion)	Total planned budget for all work	Scope baseline in dollars
EAC (Estimate at Completion)	Forecast of total project cost	Adjusts for current performance
ETC (Estimate to Complete)	Forecast of remaining cost	EAC – AC
VAC (Variance at Completion)	BAC – EAC	Predicts final over/under budget

Example forecast (from example above):

If CPI remains 0.8 → $EAC = BAC / CPI = 700 / 0.8 = **875**$ (project will exceed budget by \$175)

Summary Table – EVM & Triple Constraints

If you see...	Constraint affected	Action required
$SPI < 1$	Time	Fast-track, add resources, reduce scope
$CPI < 1$	Cost	Reduce costs, improve productivity, request budget increase
$EV < PV$ but $EV = AC$	Both time and cost	Scope may be misunderstood (re-baseline)
$EV = PV$, $CPI = 1$	All three	Perfect tracking – rare in software projects

Assignment Question 4

Estimate at Completion (EAC), Estimate to Complete (ETC), BAC vs. EAC, Variance at Completion (VAC)

1. Budget at Completion (BAC) – The Baseline

Definition:

BAC is the **total approved budget** for the project (or work package) at the start. It is derived from the **Work Breakdown Structure (WBS)** — summing up all activity cost estimates.

From Chapter 3, page 113:

“BAC – Initial budgeted costs of all work packages. It is derived from WBS → Work → Packages → Activities → Resources assigned → Durations estimated → Costs per Resources → Work Package Costs.”

Formula:

BAC = Sum of all activity budgets

Example:

A project has 5 activities, each budgeted 1,000 ETB

BAC = 5,000 ETB

Characteristics:

- Fixed at project start (unless formal re-baseline approved)
- Serves as the **scope baseline in monetary terms**
- Does **not change** with actual performance

2. Estimate at Completion (EAC) – Forecasted Total Cost

Definition:

EAC is the **forecasted total cost** of the project upon completion, calculated at a specific point in time based on **past performance and current conditions**.

From Chapter 3, page 114:

“Estimate the funds required for the complete work of the project at a given point in time.”

Four Common EAC Formulas (used in different scenarios):

Scenario	Formula	When to use
1. Original estimates are still valid	$EAC = AC + (BAC - EV)$	Current variances are one-time only
2. Cost performance will continue	$EAC = BAC / CPI$	Most common — assumes future cost efficiency same as past
3. Cost & schedule both affect budget	$EAC = AC + (BAC - EV) / (CPI \times SPI)$	Project is behind schedule AND over budget
4. Original estimates are completely flawed	$EAC = AC + ETC$ (bottom-up)	Re-estimate all remaining work from scratch

Detailed Example (Scenario 2 – Most Common)

Project: Build a mobile app

- $BAC = 100,000$ ETB
- $CPI = 0.8$ (80% cost efficiency)

Calculation:

$$EAC = BAC / CPI = 100,000 \text{ ETB} / 0.8 = \mathbf{125,000 \text{ ETB}}$$

Interpretation:

If cost performance does not improve, the project will finish **25,000 ETB over budget**.

Step-by-step logic:

1. Past performance shows we spend 1.25 ETB for every 1.00 ETB of work completed ($CPI = 0.8$)
2. Remaining work ($BAC - EV$) will likely continue at same efficiency
3. Therefore, total cost = actual spent + (remaining work / CPI)

3. Estimate to Complete (ETC) – Remaining Cost Forecast

Definition:

ETC is the forecasted **cost to complete all remaining project work** from the current point forward.

“Estimate the funds required for the remaining work of the project at a given point in time.”

Two Common ETC Formulas:

Approach	Formula	When used
Based on past performance	$ETC = (BAC - EV) / CPI$	Future cost efficiency same as past
Bottom-up (new estimate)	ETC = sum of new estimates for remaining work	Original estimates are invalid

Example (using past performance method)

Data:

- $BAC = 100,000 \text{ ETB}$
- $EV = 40,000 \text{ ETB}$ (40% of work completed)
- $CPI = 0.8$

Calculation:

$$ETC = (BAC - EV) / CPI$$

$$ETC = (100,000 - 40,000) / 0.8$$

$$ETC = 60,000 / 0.8 = \mathbf{75,000 \text{ ETB}}$$

Interpretation:

We need **75,000 ETB more** to finish the remaining 60% of work.

Alternative direct ETC:

$$EAC = 125,000 \text{ ETB (from earlier), } AC = 50,000 \text{ ETB (example)}$$

$$ETC = EAC - AC = 125,000 - 50,000 = \mathbf{75,000 \text{ ETB } \checkmark}$$

4. BAC vs. EAC – Key Differences

Aspect	BAC	EAC
Definition	Original approved budget	Forecasted total cost
When set	At project start	Updated periodically during project
Changes?	Only through formal re-baseline	Continuously updated
Reflects	Planned cost	Actual + predicted cost
Used for	Performance baseline	Funding forecasts

Comparison Scenarios:

Condition	Relationship	Meaning
On budget	$EAC = BAC$	Perfect tracking (rare)
Over budget	$EAC > BAC$	Negative variance – additional funds needed
Under budget	$EAC < BAC$	Positive variance – surplus

Example Comparison Table:

Metric	Start of project	Month 3	Month 6
BAC	100,000 ETB	100,000 ETB	100,000 ETB
EAC	100,000 ETB (assumed)	110,000 ETB	125,000 ETB
Variance	0 ETB	-10,000 ETB	-25,000 ETB

Conclusion: By month 6, the project is forecast to exceed budget by 25,000 ETB.

5. Variance at Completion (VAC) – Final Over/Under Budget

Definition:

VAC is the **expected difference** between the original budget (BAC) and the forecasted total cost (EAC).

From Chapter 3, page 114:

“Differences between Budgeted Cost & Estimated Cost (Variance at Completion)”

Formula:

$$\text{VAC} = \text{BAC} - \text{EAC}$$

Interpretation Table:

VAC Value	Meaning	Status
Positive (+X ETB)	Under budget	Good
Negative (–X ETB)	Over budget	Bad
Zero	Exactly on budget	Perfect

Example Calculations:

Scenario A – Over budget:

BAC = 100,000 ETB, EAC = 125,000 ETB

VAC = 100,000 – 125,000 = **–25,000 ETB** (need extra 25,000 ETB)

Scenario B – Under budget:

BAC = 100,000 ETB, EAC = 90,000 ETB

VAC = 100,000 – 90,000 = **+10,000 ETB** (save 10,000 ETB)

Scenario C – On budget:

BAC = 100,000 ETB, EAC = 100,000 ETB

VAC = 0 ETB

6. Complete Worked Example

Project: Software testing phase

Metric	Value
BAC	50,000 ETB
EV (at month 2)	20,000 ETB
AC (at month 2)	25,000 ETB
CPI	$20,000 / 25,000 = 0.8$

Calculations:

Term	Formula	Result
EAC	$BAC / CPI = 50,000 / 0.8$	62,500 ETB
ETC	$EAC - AC = 62,500 - 25,000$	37,500 ETB
VAC	$BAC - EAC = 50,000 - 62,500$	-12,500 ETB

Interpretation:

- Project will cost **62,500 ETB** instead of 50,000 ETB
- Need **37,500 ETB more** to finish
- Final **over budget by 12,500 ETB**

7. Summary Cheat Sheet

Term	Formula	What it tells you
BAC	Fixed	Original total budget
EAC	BAC / CPI (or other)	Forecasted final cost
ETC	$EAC - AC$	Remaining cost needed
VAC	$BAC - EAC$	Expected over/under at end

Rule of thumb :

$CPI < 1 \rightarrow EAC > BAC \rightarrow VAC \text{ negative} \rightarrow \text{Project will finish over budget}$

Assignment 5

Quality Management Report: Seven QC Tools, Statistical Sampling, Maturity Models, Adaptive vs. Predictive, and Quality Management Plan

1. The Seven Quality Control Techniques

Scholars (Kaoru) stated that "95% of quality problems can be solved with seven fundamental tools."

- **Cause-and-Effect Diagram (Fishbone Diagram)**
 - Visual tool to explore all potential factors causing a problem
 - Start with the effect on the right, list causes in categories (people, process, technology, environment)
 - From slide 17: *"Used to identify the root cause of a problem"*
- **Check Sheet (Tally Sheet)**
 - Used to collect and analyze data in real-time
 - From slide 19: example tracking media source of complaints – most complaints via text message, more on Monday/Tuesday
- **Pareto Chart**
 - Based on 80-20 rule (80% of problems come from 20% of causes)
 - From slide 21: user complaints about login issues → fix login first
- **Flowchart**
 - Graphic display of process logic and flow
 - From slide 23: testing tasks in SDLC – unit → integration → system → acceptance testing
- **Histogram**
 - Shows distribution of data (e.g., defects per software module)
 - Helps visualize which components have the most bugs
- **Control Chart**
 - Monitors process stability over time
 - Upper/lower control limits determine if process is "out of control"

- **Scatter Diagram**

- Shows relationship between two variables (e.g., lines of code vs. defects)
 - Helps identify correlations
-

2. Statistical Sampling Techniques for Quality Control

Statistical sampling = selecting a portion of work products for inspection rather than examining everything. Saves time and resources while maintaining confidence.

- **Random Sampling** – Every item has equal chance of selection
- **Stratified Sampling** – Divide into layers (UI, backend, database) and sample from each
- **Attribute Sampling** – Classify as conforming or non-conforming (pass/fail)
- **Variable Sampling** – Measure actual values (response time, memory usage)

From Chapter 4, slide 25 (Testing Tasks): Statistical sampling is used in unit testing, integration testing, system testing, and user acceptance testing.

3. Maturity Models

3.1 Software Quality Function Deployment (QFD) Model

- Translates customer requirements into technical specifications
- Ensures quality is built from the beginning
- Key components: customer requirements, technical requirements, relationship matrix
- Benefit: Reduces misalignment between stakeholder expectations and delivered features

3.2 Capability Maturity Model Integration (CMMI)

Source: Chapter 4, slide 6

Five maturity levels:

- **Level 1 – Initial:** Ad hoc, chaotic processes
- **Level 2 – Managed:** Basic project management (requirements, planning)
- **Level 3 – Defined:** Standardized processes across organization
- **Level 4 – Quantitatively Managed:** Statistical control using metrics
- **Level 5 – Optimizing:** Continuous process improvement

Impact on quality: Higher maturity levels = lower defect rates, better predictability, improved stakeholder satisfaction.

3.3 Project Management Maturity Models (PMMM)

- Assesses organization's ability to deliver projects successfully
- Components: Knowledge (what to do), Measurement (how well), Improvement (getting better)
- Relevance: Mature organizations consistently apply quality planning, assurance, and control

3.4 Six Sigma

- Data-driven methodology for eliminating defects
- Target: 3.4 defects per million opportunities

DMAIC Framework:

- **Define** – Identify quality problem (e.g., high bug rate)
- **Measure** – Collect baseline data (current defect density)
- **Analyze** – Find root causes (insufficient testing, no code reviews)
- **Improve** – Implement solutions (add mandatory reviews, increase test coverage)
- **Control** – Sustain gains (monitor defect density monthly)

Table 1: Comparison of Maturity Models

Model	Focus	Key Output
QFD	Requirements translation	Quality function deployment matrix
CMMI	Process maturity	Maturity level certification
PMMM	Project management capability	Process improvement roadmap
Six Sigma	Defect reduction	DMAIC projects, statistical control

4. Adaptive vs. Predictive Models in Quality Management

Predictive Model (Waterfall):

- Heavy upfront quality planning

- Phase-end reviews and process audits
- Final system testing phase (late defect detection)
- Formal change control board (from Chapter 2)
- Extensive documentation (IEEE 730)
- Low customer involvement (milestones only)
- Best for: stable requirements, safety-critical systems

Adaptive Model (Agile/Scrum):

- Just-in-time quality planning per sprint
- Daily standups, sprint reviews, retrospectives
- Continuous testing (unit, integration, acceptance each sprint)
- Changes accepted in next sprint backlog
- Lightweight documentation (user stories, acceptance criteria)
- High customer involvement (every sprint review)
- Best for: uncertain requirements, fast-changing markets

"Modern quality management prefers prevention to inspection." Agile embodies this by building quality into each iteration rather than inspecting at the end.

5. Developing a Quality Management Plan for a Group Project

5.1 Quality Policy and Objectives

- Deliver software with $\leq 2\%$ critical defects
- 100% conformance to written specifications
- 95% user satisfaction

5.2 Quality Standards to Be Used :

- **ISO 9001** – Quality management system framework
- **IEEE 730** – Software Quality Assurance Plan structure
- **CMMI Level 3** – Standardized processes across the team

5.3 Quality Planning Activities

- Define defect rate targets (<5 per 1000 lines of code)
- Set response time requirements (<2 seconds for 95% of transactions)
- Select tools: Jira for bug tracking, Selenium for automated testing

5.4 Quality Assurance Activities

- Weekly code reviews
- Process audits every two weeks
- Test coverage reports (minimum 80%)
- Lessons learned documentation after each iteration

5.5 Quality Control Activities (from Ch.4, slide 15-24)

- **Unit testing** – After each module (JUnit, PyTest)
- **Integration testing** – Weekly (Postman, Jenkins)
- **System testing** – Before each release (Selenium)
- **User acceptance testing** – End of each iteration (stakeholder demo)

5.6 Using the Seven QC Tools in the Quality Plan

- **Fishbone** – Root cause analysis for any critical bug
 - **Check sheet** – Daily defect logging by tester
 - **Pareto chart** – Prioritize which bug categories to fix first
 - **Flowchart** – Map CI/CD deployment pipeline
 - **Histogram** – Show defect distribution by software module
 - **Control chart** – Monitor defect rate across sprints
 - **Scatter diagram** – Correlation between code changes and new defects
-

Table 2: Sample Quality Management Plan Structure

Section	Content
Purpose	Ensure final software meets requirements and satisfies users
Scope	All development, testing, and deployment activities
Quality objectives	<2% critical defects, 95% user satisfaction
Standards	ISO 9001, IEEE 730, CMMI Level 3
QA activities	Code reviews, process audits, retrospectives
QC activities	Unit, integration, system, UAT testing
Seven QC tools	Fishbone, check sheet, Pareto, flowchart, histogram, control chart, scatter diagram
Metrics	Defect density, test coverage, MTDD, MTTR

Generally

"Software project quality management is a proactive process. Quality is not optional; it directly impacts project timelines, costs, and success. High-quality software results in fewer defects, lower costs, faster delivery."

Key Takeaways:

- Seven QC tools address 95% of quality problems
- Statistical sampling reduces inspection effort while maintaining confidence
- Maturity models (CMMI, QFD, Six Sigma) provide structured improvement paths
- Adaptive models (Agile) = continuous quality; Predictive models (Waterfall) = phase-end quality
- A Quality Management Plan must include standards, tools, activities, metrics, and responsibilities

References

1. Course Lecture Slides. (2026). *Project Management (CoSc1012)* – Chapters 1, 2, 3, 4, 5.
 2. Project Management Institute (PMI). (2013). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* (5th ed.).
 3. Schwalbe, K. (2019). *Information Technology Project Management* (9th ed.). Cengage Learning.
 4. Hughes, B., & Cotterell, M. (2009). *Software Project Management* (5th ed.). McGraw-Hill.
 5. Software Engineering Institute (SEI). *Capability Maturity Model Integration (CMMI)*.
 6. International Organization for Standardization (ISO). *ISO 9001: Quality Management Systems*.
 7. IEEE Computer Society. *IEEE 730 – Software Quality Assurance Plans*.
 8. Beck, K., et al. (2001). *Manifesto for Agile Software Development*.
 9. Montealegre, R., & Keil, M. (2000). Denver International Airport case study. *MIS Quarterly*, 24(3).
-